Gantry Team

Your design is now at the software level: operator, app, controller, sensors, motor. The E-stop goes straight to the motor, not through the controller. Great!

Things to think about:

- The E-stop must work even if the app or controller freezes. Make it a physical button the operator presses, not something triggered through the app.
- Make arrow labels specific: "target (x, y)", "position, limit hit, weight", "move command", "ready / fault".

Today's activity

Your design is Version 1. Decide what to build first, and with what:

- **Pick one path:** the boxes your first CORE item goes through, from start to end.
- **For each box:** where it runs (phone, browser, laptop, cloud, device), what you build it with (language, framework, tool), and why.
- **Name your riskiest piece:** the part you're least sure will work.

Pick tools you know or can learn in time. Libraries are fine; the CORE logic is yours to build.

Our first CORE item: Routing/Movement, built first as a simulation: the operator gives a target (x, y) and a Fusion 360 model of the gantry moves the hook there, staying inside the workspace limits. Sizing: about 1 cm position accuracy with a small load of a few pounds.

Our path (boxes in order): Operator → App [“target (x, y)”] → Controller [“move command”] → Gantry model (motors), then Sensors [“position, limit hit, weight”] → Controller [“ready / fault”] → App. The Controller refuses any target until homing is done. For Version 0 the only input is the browser app (Bluetooth and joystick come later). The E-stop is a separate input that bypasses the Controller: in the simulation it is a stop button that freezes the model, and on hardware it will be a physical button that cuts motor power directly, so it works even if the Controller freezes.

Box on the path	Where it runs	Built with	Why
Operator	Browser on a phone or laptop	Clicking a point on the workspace map or typing a target (x, y) in the app	The operator only needs to send a target (x, y). Entering it in the app lets the software check it before anything moves.
App	Browser (works on phone and laptop)	HTML/CSS/JavaScript with a canvas map of the workspace; WebSocket to the controller	Runs on both devices with no install. Checks the target is inside the workspace, sends "target (x, y)", and shows "ready / fault".
Controller	Laptop for now; STM32 microcontroller later (chip not selected yet)	Python program with our own logic: “not homed” lockout and homing, soft limits, converting targets to move commands, safety decisions from sensor readings, fault handling	The Controller is the one box that makes safety decisions. Python lets us build and test that logic before choosing the chip and matches Fusion 360's API. We will port it to C on the STM32.

Gantry model (motors)	Laptop: Fusion 360	Fusion 360 CAD assembly with X and Y slider joints plus the lead-screw hook lift, moved by a Fusion 360 Python add-in; motion study and stress simulation of the frame	Shows movement on our real design, so we can check reach, travel, and collisions. Also checks frame deflection under a few-pound load before we buy parts.
Sensors	Laptop: simulated in the controller and add-in	Simulated limit switches at each axis end (for homing), motor encoder position read from the model's joint values, and a weight sensor value	Sensors only send readings; the Controller decides. Lets us test homing, limit hits, and overload cases before hardware exists.

Our riskiest piece is linking our controller logic to the Fusion 360 model so the gantry actually moves in response to commands. Fusion 360 is a CAD tool, not a real-time simulator, so driving its joints from outside through its Python API may be slow or awkward.

We'll test it by building a small test first: a Fusion 360 add-in that reads a hard-coded list of (x, y) targets and moves the gantry joints to each one. Next we connect it to the controller and send targets from the browser, including one sent before homing and some outside the workspace. The test passes if the model reaches each valid target within 1 cm, and the before-homing and out-of-bounds targets are refused with a fault. If the live link is too slow, we will keep Fusion 360 for design and load checks and show live movement in a 2D view in the browser.

Something new we need to learn (if any): the Fusion 360 Python API (add-ins and controlling joints), WebSocket communication between a browser and a Python program, and later programming an STM32 in C (STM32CubeIDE/HAL) once we select the chip.